

Plan-2: Task Statement

AI-Assisted Product Planning for the Academic AI Compute Fabric

Course Activity pairs (4–6 Hours) **AI-Assisted (BMad & WDS Workflows)**

Deliverable: Complete Module Planning Package (PRD, UX Design, Architecture, Readiness Gate, and AI Decision Log).

1. Executive Summary & Project Framing

The Shared Scenario: Academic AI Compute Fabric

A university is expanding its local artificial intelligence infrastructure for teaching, laboratory coursework, student experimentation, and faculty research. The institution owns a heterogeneous on-premise compute fleet consisting of: * **31 Workstations:** Each equipped with an NVIDIA GPU with **24 GB VRAM** (e.g., RTX 4090 / RTX 3090). * **1 Central Server:** Equipped with **two 48 GB GPUs** (e.g., RTX 6000 Ada / A40, totaling 96 GB VRAM).

At present, access to these machines and model endpoints is fragmented, manually coordinated via spreadsheets and chat channels, and prone to resource contention. The university requires a modern management platform to govern controlled access to local Large Language Models (LLMs), schedule training workloads, enforce subject-level quotas, manage GPU reservations, and provide observability across the fleet.

In realistic enterprise environments, **Kubernetes** exposes NVIDIA GPUs as schedulable capacity via device plugins, **vLLM** provides high-throughput serving with Prometheus-compatible token metrics, and **Kubeflow's Training Operator** orchestrates distributed ML jobs (e.g., PyTorchJob).

Each student pair works exclusively on **ONE standalone subsystem module** (assigned from the Annex). Your project must operate completely independently using local fixtures, seed data, mock APIs, or simulated GPU telemetry.

What You DO NOT Need to Do:

- You do not need to integrate with another team's code.
- You do not need to share a database or coordinate distributed migrations.
- You do not need to negotiate cross-team API contracts for this task.
- You do not need to deploy a shared Kubernetes cluster across groups.
- You do not need to participate in a monolithic, multi-service joint demo.

What You MUST Prove:

- Prove that your assigned subsystem is **semantically coherent, mathematically sound, defensively specified, and independently testable**.
- Demonstrate mastery over the AI council (**BMad + WDS**) by directing specialized personas to transform raw intent into correct engineering specifications.
- Provide verifiable evidence that human engineering judgment directed and audited every AI-generated decision.

2. Case Study Sources & Baseline References

Review the following reference materials before starting Phase 1:

Source Document	Core Content & System Context
project - university_ai_compute_management_platform_brief.pdf	Institutional mission, stakeholder tensions, heterogeneous hardware fleet profile, and high-level platform goals.
Fleet Topology Profile	31 nodes: 1 GPU (24 GB VRAM, 16–32 vCPU, 64 GB RAM, 500 GB NVMe). 1 server: 2 GPUs (48 GB VRAM each, 64 vCPU, 256 GB RAM, 4 TB NVMe).
Technology Baseline	Linux, Docker Compose / single-node K8s (Kind/K3s), Python (FastAPI) or TypeScript (Node), simulated Prometheus/OpenTelemetry streams.
BMad & WDS Skill Guides	Personas for Product (bmad-agent-pm) UX (wds-agent-freya-ux) Architecture (bmad-agent-architect) Testing (bmad-tea) Review (bmad-review)

3. Learning Objectives

By completing Plan-2, your team will be able to:

1. **Extract & Scope a Module PRD:** Isolate the functional capabilities, business rules, and strict operational constraints of your subsystem from an institutional narrative without scope creep.
 2. **Design Grounded UX Journeys:** Craft concrete, multi-state interface specifications with named academic protagonists (students, professors, lab admins) using WDS workflows.
 3. **Establish Inviolable Architectural Invariants (AD-n):** Formulate durable technical rules with strict Status, Binds, Prevents, Rule, and Trade-off clauses that prevent silent system decay.
 4. **Enforce Pre-Implementation Quality Gates:** Subject specifications to multi-lens adversarial review to compile an unambiguous readiness sign-off.
 5. **Practice Disciplined AI Governance:** Command AI agents as subordinate technical specialists, systematically recording prompts, rejecting invalid outputs, and proving human-in-the-loop validation.
-

4. Standard AI-Assisted Workflow

All groups must follow this rigorous, specification-first sequence:

[1. Understand & Scope] → [2. PRD Specification] → [3. UX Scenarios & Surfaces] →
[4. System Architecture] → [5. Readiness Gate]

1. **Understand Before Prompting:** Analyze your assigned module in the Annex. Identify boundary ambiguities, document explicit assumptions, and list non-goals.
 2. **Write the Specification Before Code:** Define user journeys, measurable NFRs, state machines, domain policies, and error models prior to any implementation tasks.
 3. **Design Before Execution:** Challenge the AI to present 2–3 architectural alternatives, critically evaluate trade-offs, select the optimal design, and justify your choice.
 4. **Validate Adversarially:** Never trust AI outputs by default. Run adversarial and edge-case review lenses to surface race conditions, security flaws, and untestable claims.
 5. **Maintain an Audit Trail:** Document key prompts, agent iterations, and at least three instances where your team **rejected, corrected, or substantially improved** an AI recommendation.
-

5. Common Constraints Across All 12 Modules

To ensure fair assessment and technical comparability across disparate challenges, all modules share these non-negotiable constraints:

1. **Independent Executability:** The subsystem must be runnable as a standalone unit using Docker Compose or a clean local virtual environment.
 2. **Hardware Agnosticism:** Physical NVIDIA GPUs are **not required**. Systems must accept seed fixtures, simulated GPU telemetry, mock driver metrics, or virtual node states.
 3. **Non-Trivial Decision Logic:** According to the module needs, the solution cannot be a simple CRUD wrapper. It must implement an explicit algorithm, policy evaluation, optimization planner, or state-machine.
 4. **Algorithmic Explainability:** Every automated decision (e.g., job rejection, placement plan, quota throttling, or access denial) must produce, human-readable, cited justifications.
 5. **Mandatory Non-Happy Paths:** Specifications and prototypes must design for at least two failure states (e.g., out-of-memory, timeout, quota breach, conflict, or node failure).
 6. **Defensive Contract Format:** All architectural decisions must adhere strictly to the BMad Architectural Invariant (AD-n) format.
 7. **Traceability & AI Audit Log:** The submission must include an **AI Decision Log** with documented prompts, including rejected AI proposals.
-

Detailed Phase-by-Phase Instructions

Phase 1 — PRD & Capability Scoping (60–75 Min)

Goal

Produce a comprehensive prd.md scoped strictly to your assigned module. The document must define user value, functional requirements (FRs), non-functional requirements (NFRs) with explicit numeric metrics, and system boundaries.

Instructions

1. Activate bmad-prd (or converse with **John**, bmad-agent-pm).
2. Provide the university brief (project - university_ai_compute_management_platform_brief.pdf) and your module profile from the Annex with the spec.md and project-brief.md
3. Establish module boundaries: Clearly specify what your module owns and what it mocks/delegates. Declare out-of-scope capabilities explicitly.

4. Define your named academic protagonists: Avoid generic “the user.” Introduce specific personas (e.g., *Dr. Aris Thorne, NLP Professor; Mateo, Undergraduate Student; Elena, AI Lab Operations Engineer*).
5. Formulate Functional Requirements (FRs): For each capability, define:
 - System responsibility (what the system guarantees).
 - Exact trigger, inputs, validation rules, and outputs.
 - Measurable testable condition.
6. Formulate Non-Functional Requirements (NFRs): Enforce numbers and explicit tolerances (e.g., “*Policy evaluation completes under 50 concurrent requests*”; “*Zero leakage of authentication tokens in debug logs*”).
7. **Review Gate:** Run bmad-review with the adversarial lens:

```
/bmad-review docs/planning-artifacts/prd.md --lens adversarial
```

Triage every finding: **Accept** (fix in PRD), **Defer** (post-V1), or **Reject** (document why the AI critique is invalid).

Deliverables — Phase 1

2. prd.md (complete, scoped to your module)
 3. review-prd-adversarial.md (review report with triaged responses)
 4. Phase 1 entries in your **AI Decision Log**
-

Phase 2 — UX Design, Scenarios & Surfaces (60–75 Min)

Goal

Produce the visual design spine (DESIGN.md), behavioral experience spine (EXPERIENCE.md), 4 structured user scenario outlines, and detailed page/surface specifications for your standalone prototype.

Part A — Design & Experience Spines with bmad-ux

1. Activate bmad-ux (or converse with **Sally / Freya**, wds-agent-freya-ux).
2. Declare target form factors and interfaces:
 - Administrative/Observability modules: Desktop-first, data-dense, monospace code/telemetry views.
 - Student/Reservation/Chat modules: Responsive web, clear action states, friction-free forms.
 - Headless/Engine modules (Gateway, Policy Engine): Developer console, interactive API explorer (Swagger/OpenAPI), and CLI dashboard.
3. Author DESIGN.md: Color tokens, typography, hierarchy, and status color states (e.g., Available: Green, Reserved: Amber, Active/Busy: Blue, Fault/Offline: Red).

4. Author EXPERIENCE.md: Loading, Empty, Error, and Success states; explainability banners for denied/deferred actions; accessibility standards (WCAG 2.1 AA).

Part B — Scenarios & Page Specs with wds-3-scenarios & wds-4-ux-design

1. Run wds-3-scenarios to structure the **4 primary scenario outlines** required for your module (see Annex).
2. For each scenario, run the design loop in wds-4-ux-design:
 - [C] Conceptualize: Layout wireframes and key screen elements.
 - [P] Write Specifications: Detailed component specs, spacing, button triggers, and dynamic feedback.
 - [V] Validate: Ensure every error state, rejection banner, and edge condition is visually addressed.
3. Produce at least **two key-screen wireframes/mocks** (ASCII diagram, Mermaid UI layout, or SVG/PNG asset).
4. **Review Gate:** Run bmad-review across UX artifacts:

```
/bmad-review docs/ux/ --lens edge-case-hunter
```

Deliverables — Phase 2

5. DESIGN.md & EXPERIENCE.md
6. C-UX-Scenarios/00-ux-scenarios.md + 4 scenario outline files
7. Per-page specifications for all core surfaces
8. Key-screen wireframes / UI mockups
9. review-ux-edge-cases.md with triaged decisions

Phase 3 — Technical Architecture & Invariants (60–80 Min)

Goal

Produce ARCHITECTURE.md establishing the internal components, data schemas, telemetry integration points, and machine-enforceable Architectural Invariants (AD-n).

Instructions

1. Activate bmad-architecture (or converse with **Winston**, bmad-agent-architect).
2. Provide prd.md, EXPERIENCE.md, and your hardware/mock constraints.
3. Define the Core Architecture:
 - **Component Topology:** external boundaries including API Gateway, communication, notifications, messaging if needed.
 - **Data Model & Schemas:** High level entities and context.

- **API / Event Contracts:** High level API routes and communication definitions.
 - **Mermaid Architecture Diagrams:** At minimum, a Context Diagram and one High-Level Component Interaction Diagram illustrating the core non-trivial algorithm.
4. **Review and refine Architectural Invariants (AD-n):** Every invariant must adhere to the mandatory 5-part structure:
- **Status:** [ADOPTED]
 - **Binds:** Specific submodules, routers, database models, or workers.
 - **Prevents:** The exact architectural bug, race condition, data corruption, or unauthorized leak permanently blocked.
 - **Rule:** Precise technical rule written with ZERO elastic words (no “efficiently”, “appropriately”, “securely”).
 - **Trade-off:** What is explicitly sacrificed, deferred, or limited to guarantee this invariant.
5. **Review Gate:** Run both adversarial and edge-case reviews:

```
/bmad-review docs/architecture/ --lens adversarial
/bmad-review docs/architecture/ --lens edge-case-hunter
```

Deliverables — Phase 3

10. ARCHITECTURE.md (complete with AD-n invariants)
11. Mermaid diagrams (Context and Components diagrams)
12. review-arch-adversarial.md & review-arch-edge-cases.md with documented triage

Phase 4 — Implementation Readiness Gate Check (20–30 Min)

Goal

Execute the formal pre-implementation quality gate to verify that all planning artifacts are 100% aligned, self-consistent, and free of blocking contradictions before coding begins.

Instructions

-
1. Activate `bmad-check-implementation-readiness`.
 2. Provide all four finalized artifacts.
 3. Before running this check, ensure **all review findings from phases 1, 2, and 3 have been triaged** — the readiness check will flag untriaged findings as blockers.
 4. **Evaluate the Gate Status:**
 - **PASS:** Specifications certified 100% complete and consistent.
 - **CONCERNS:** Minor non-blocking gaps; proceed only with documented mitigation.

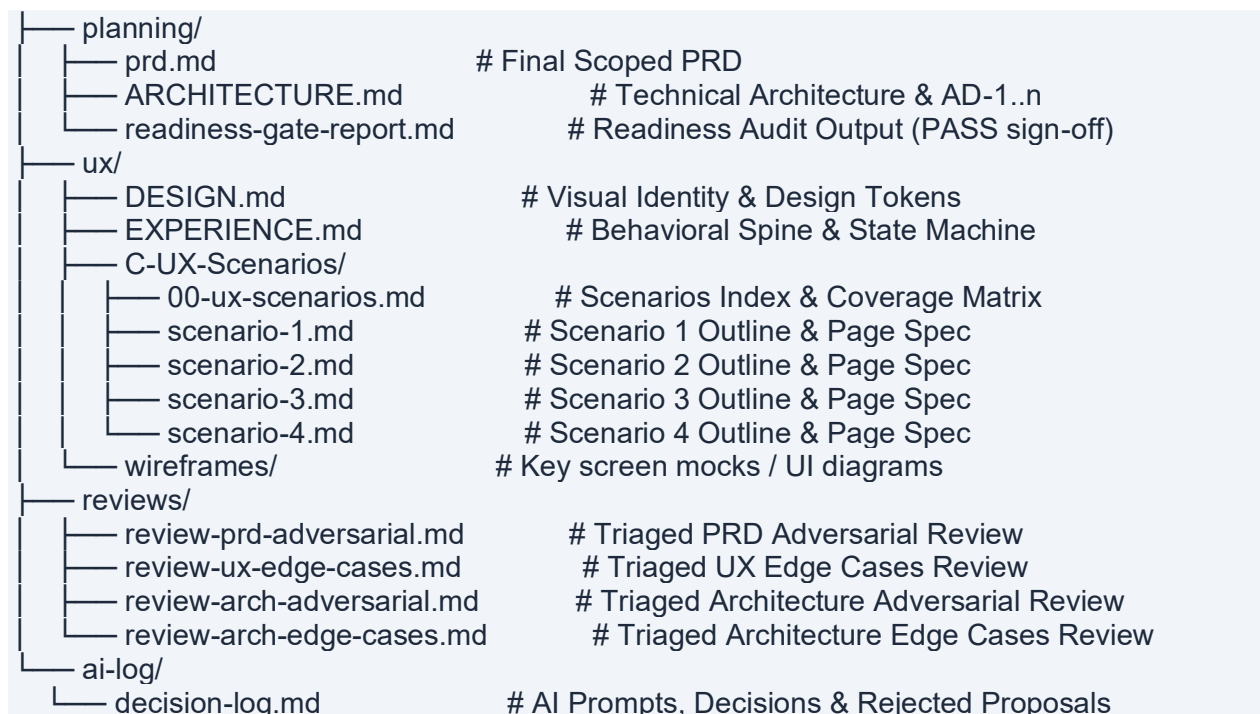
- **FAIL:** Critical collision detected; must refactor conflicting clauses before sign-off.
5. Capture the terminal readiness audit report and complete the sign-off block.

Deliverables — Phase 4

readiness-gate-report.md (full terminal output, blocker resolution log, and formal sign-off)

6. Submission Package Checklist

Your team repository should contain the following standardized folder structure and files upon submission:



AI Decision Log Requirements (decision-log.md)

1. **Significant Entries:** Document moments where human judgment directed, corrected, or constrained the AI.
2. **Rejected AI Proposals:** Specifically highlight at least three suggestions generated by the AI that your team **rejected, corrected, or substantially improved**, explaining your technical rationale.

7. Assessment & Grading Rubric

Criterion	Weight	Outstanding (4)	Proficient (3)	Developing (2)	Beginning (1)
-----------	--------	-----------------	----------------	----------------	---------------

Criterion	Weight	Outstanding (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Scoping & PRD Rigor	20%	Scoped strictly to module boundaries; all 4 scenarios traced to FRs; NFRs testable with explicit numbers; named academic protagonists throughout; zero vague elastic language.	Scoped to module; minor NFR metric omissions; 3–4 scenarios traced; protagonists present but occasional generic terms.	Scope leaks into other modules; several untestable NFRs; abstract user roles (“the user”); vague requirements.	PRD is a generic copy of the system brief; untestable; missing functional depth.
UX Design & Scenario Quality	20%	DESIGN.md and EXPERIENCE.md complete; all 4 screen states (Loading/Empty/Error/Success) specified; explainability for rejections designed; clear wireframes; WCAG accessibility addressed.	Both spines present; minor state omissions (e.g., missing empty state); wireframes present; solid UX flow.	One spine thin or missing; wireframes unclear; error states and explainability banners neglected.	Spines missing or generic; no screen specifications; purely abstract text.
Architecture & Invariants (AD-n)	20%	6–8 robust AD-n invariants with complete Binds/Prevents/Rule/Trade-off; security boundaries and data models explicit; valid Mermaid diagrams; explainability architecture clear.	5–6 AD-n invariants; 1–2 have minor vague phrasing; data model and component diagrams present; solid technical reasoning.	No invariants or formulated as generic tips; diagrams missing or uninformative; data model incomplete.	Invariants absent; architecture is an ungrounded high-level essay without technical contracts.
Readiness Gate	10%	Implementation readiness check executed with verified PASS and clean triage.	Readiness check executed with minor unresolved concerns documented.	Readiness check reports unhandled failures.	Readiness check was never run or ignored.
AI Governance & Decision Log	15%	More than 7 detailed entries demonstrating proactive human leadership; Several documented AI rejections with deep technical reasoning; clear proof that human judgment	6–7 entries; documented AI rejections; good evidence of critical evaluation and oversight.	4–5 entries; passive acceptance of most AI recommendations; weak rationale for accepted suggestions.	No documented entries; uncritical copy-paste of raw AI output; zero evidence of oversight.

Criterion	Weight	Outstanding (4)	Proficient (3)	Developing (2)	Beginning (1)
guided every step.					
Adversarial Review & Triage	15%	All 4 specialized review reports executed; every single finding triaged with clear Accept/Defer/Reject justification; accepted findings visibly fixed in artifacts.	All reviews executed; most findings triaged with reasonable explanations; key blockers addressed.	Partial reviews run (2–3); superficial triage; several critical warnings left unaddressed.	Reviews omitted or findings ignored without triage.

Annex: The 12 Independent Engineering Modules

Each team is assigned **one** subsystem from the matrix below. Every module represents a realistic, mission-critical component of an enterprise academic compute cluster.

Module 1: User, Role, and Subject Access Manager

13. **Tier:** Foundation
14. **Core Problem:** The university serves thousands of students across dozens of courses (e.g., CS101, NLP Advanced, Computer Vision). System administrators cannot manually provision API keys or node logins. The platform requires a localized identity, role, and academic subject entitlement system that maps students and instructors to approved courses and permission tiers.
15. **Named Protagonists:** *Dr. Marcus Vance (CS Department Chair & Admin); Sofia (Teaching Assistant for Intro to ML).*
16. **4 Mandatory Scenarios:**
 1. *Course Roster Ingestion:* Admin imports a semester roster mapping 120 students to CS-480 with role Student.
 2. *Privilege Escalation Request:* Teaching assistant requests temporary Instructor-Privileged role to test model training.
 3. *Revocation upon Course Drop:* Student drops a course; system revokes access to the subject's private model pool within 60 seconds.
 4. *Audit Query:* Auditor inspects all users granted access to restricted research models during the previous 30 days.
17. **Standalone Prototype:** Web Admin Portal / RBAC API Explorer with mock identity fixtures.
18. **Mandatory Edge Case:** Simultaneous membership in multiple courses with conflicting permission tiers (e.g., student in CS101 with 0 GPU access, but TA in CS480 with high priority).

Module 2: Academic GPU Policy Engine

19. **Tier:** Intermediate
 20. **Core Problem:** Static permissions fail when campus conditions change. GPU access must evaluate dynamic policies based on subject enrollment, user role, current time, laboratory class schedules, requested GPU type (24 GB vs. 48 GB), and model weight class. Every evaluation must return an immediate, explainable ALLOW, DENY, or CONSTRAIN decision.
 21. **Named Protagonists:** *Elena (Undergraduate in Computer Vision); Dr. Rebecca Reed (Lab Director).*
 22. **4 Mandatory Scenarios:**
 1. *Scheduled Lab Protection:* System automatically denies background student jobs during scheduled lab class hours (Mon/Wed 10:00-12:00).
 2. *VRAM Tier Gate:* Student requests access to a 70B model requiring 48 GB VRAM; policy engine intercepts and rejects with an explanation of model tier restrictions.
 3. *Overnight Grace Policy:* System permits low-priority batch training after 22:00 provided no scheduled morning classes are pending.
 4. *Emergency Freeze Rule:* Lab admin activates an emergency compute freeze for an upcoming exam; all non-exam requests are denied.
 23. **Standalone Prototype:** Dynamic Policy Simulation Console with policy rule editor and interactive request evaluator.
 24. **Mandatory Edge Case:** Clock rollover / boundary edge conditions where an active job approved under night policy extends into a protected morning class window.
-

Module 3: GPU Lab Digital Twin

25. **Tier:** Intermediate
26. **Core Problem:** Hardware in physical labs fails, overheats, and disconnects. The university needs a real-time digital twin that maintains the live state of all 32 physical machines (31 workstations with 1 GPU, 1 server with 2 GPUs). It tracks VRAM occupancy, temperatures, driver health, storage free space, active reservations, and maintenance locks.
27. **Named Protagonists:** *Carlos (Hardware Lab Technician); Alex (Graduate Researcher).*
28. **4 Mandatory Scenarios:**
 1. *Fleet Heartbeat Ingestion:* Ingest simulated periodic telemetry heartbeats from 32 nodes; detect silent nodes missing > 3 intervals.
 2. *Thermal Anomaly & Automatic Cordon:* Node ws-gpu-14 reports GPU temperature > 88°C; twin marks node DEGRADED and cordons it from new jobs.
 3. *Simulated Workstation Reboot:* A lab student physically reboots machine ws-gpu-05; twin detects state loss and transitions node to RECOVERING.

4. *Maintenance Drain*: Technician schedules maintenance on the 48 GB dual-GPU server; twin manages graceful workload drain.
 29. **Standalone Prototype**: Interactive Fleet Map / Grid Dashboard with simulated telemetry injector (fault generator).
 30. **Mandatory Edge Case**: Flapping node connectivity (node transitions between online and offline every 5 seconds) without causing state oscillation or alert floods.
-

Module 4: GPU Reservation System

31. **Tier**: Intermediate
 32. **Core Problem**: When research students need uninterrupted GPU time for benchmarks or when professors need 30 machines for a 2-hour final exam, ad-hoc usage causes collisions. The system must allow users to reserve single GPUs, entire machines, or the dual-48GB server for future time windows without overlaps, enforcing strict quota and duration limits.
 33. **Named Protagonists**: *Lucas (Master's Student preparing thesis)*; *Professor Zhang (AI Exam Coordinator)*.
 34. **4 Mandatory Scenarios**:
 1. *Time-Slot Booking*: Lucas books 1 workstation for 4 hours tomorrow evening; system checks availability and locks slot.
 2. *Bulk Class Reservation*: Professor Zhang reserves 25 workstations for Friday 14:00–16:00; system reserves capacity across available nodes.
 3. *Overlap Rejection & Alternative Suggestion*: Student requests a conflicting slot; system denies request and returns the next 3 available slots.
 4. *No-Show Auto-Release*: User fails to claim reservation within 15 minutes of start time; slot is automatically cancelled and released to the standby pool.
 35. **Standalone Prototype**: Interactive Timeline / Calendar Booking UI and Reservation Conflict Engine.
 36. **Mandatory Edge Case**: Multi-resource atomic reservation where 24 of 25 nodes are available but 1 fails; reservation must abort cleanly without partial locks.
-

Module 5: Model Catalog and Entitlement System

37. **Tier**: Foundation
38. **Core Problem**: Students and faculty experiment with numerous open-weight LLMs (Llama-3, Qwen, Mistral, DeepSeek), but unvetted models pose license, security, and VRAM risks. The university needs a centralized catalog of approved models, storing weight formats (GGUF, AWQ, FP16), VRAM footprint, context lengths, and license metadata, and gating visibility by course entitlement.
39. **Named Protagonists**: *Valeria (MLOps Specialist)*; *David (Undergraduate Student in Ethics & AI)*.
40. **4 Mandatory Scenarios**:

1. *Model Registration & Validation*: Admin publishes Llama-3-8B-Instruct-Q4; system validates VRAM footprint (5.8 GB) and required compute tier.
 2. *Subject Entitlement Mapping*: Admin entitles CS-480 students to view and invoke models in the vision-language category.
 3. *License Agreement Acceptance*: Student accesses an ungated model; system displays institutional usage agreement before granting access.
 4. *Deprecation Lifecycle*: A model version is marked deprecated; existing users receive deprecation notices and replacement pointers.
41. **Standalone Prototype**: Filterable Model Catalog Portal with role-gated visibility and model profile inspect views.
 42. **Mandatory Edge Case**: Model whose minimum required VRAM exceeds workstation limits (24 GB) but fits the dual-GPU server (48 GB); catalog must restrict entitlement to privileged server users.
-

Module 6: Model Distribution Planner

43. **Tier**: Advanced
 44. **Core Problem**: Downloading a 20 GB model file over the campus network takes 15 minutes. If 30 workstations each download weights independently, network switches saturate. The system needs an intelligent distribution planner that calculates an optimal pre-caching and replication plan based on local SSD free space, historical course demand, GPU VRAM compatibility, and network bandwidth limits.
 45. **Named Protagonists**: *Dr. Isaac Bell (Systems Architect)*; *Tariq (Campus Network Engineer)*.
 46. **4 Mandatory Scenarios**:
 1. *Classroom Pre-Cache Plan*: Given tomorrow's CS101 lab needing Qwen-2.5-7B on 20 nodes, planner generates an optimal staged download schedule.
 2. *Disk Space Eviction Calculation*: Node ws-gpu-08 has only 15 GB free; planner determines which LRU (least-recently-used) model cache to safely evict.
 3. *Replication Rebalancing*: A model experiences high request queues; planner calculates placement of 4 additional replicas without exceeding node storage budgets.
 4. *Bandwidth-Constrained Plan Generation*: Under a campus network throttling rule (< 1 Gbps aggregate), planner stages transfers sequentially.
 47. **Standalone Prototype**: Algorithmic Placement Optimizer (CLI / Visual Dashboard) taking node state JSON and producing verified placement plans.
 48. **Mandatory Edge Case**: Knapsack constraint violation where no single node has sufficient disk space for a requested high-demand model, triggering multi-node partial eviction plans.
-

Module 7: Local LLM Inference Gateway

- 49. **Tier:** Intermediate
 - 50. **Core Problem:** Applications and student notebooks require an OpenAI-compatible API endpoint (/v1/chat/completions) that routes requests to underlying model workers. The gateway must authenticate student API tokens, enforce subject authorization, apply rate limits (requests per minute), reject unauthorized models, and gracefully handle worker timeouts or model crashes.
 - 51. **Named Protagonists:** *Maya (Student building chatbot project); System Router.*
 - 52. **4 Mandatory Scenarios:**
 - 1. *Authenticated Chat Request:* Maya sends a valid chat completion request; gateway validates bearer token, verifies course entitlement, and routes to mock worker.
 - 2. *Rate Limit Enforcement (429):* Student notebook loops aggressively; gateway triggers rate-limiter, returns HTTP 429 with Retry-After header.
 - 3. *Unauthorized Model Interception (403):* Student requests an unassigned research model; gateway intercepts with explanatory 403 Forbidden payload.
 - 4. *Worker Failure & Circuit Breaker (503):* Downstream mock inference worker times out (> 10 s); gateway trips circuit breaker and returns formatted service error.
 - 53. **Standalone Prototype:** FastAPI / Express API Gateway with interactive Swagger UI and real-time request tracing console.
 - 54. **Mandatory Edge Case:** Token authentication expiration mid-stream during a chunked Server-Sent Events (SSE) streaming response.
-

Module 8: Token Usage and Quota Service

- 55. **Tier:** Foundation
- 56. **Core Problem:** Inference compute is finite. Left unmonitored, a single runaway script can consume entire GPU clusters. The platform needs a quota service that meters prompt tokens, completion tokens, request counts, and estimated GPU-seconds, enforcing daily and monthly budgets allocated by student, group, or academic course.
- 57. **Named Protagonists:** *Gabriel (Financially accountable Academic Dean); Sam (Undergraduate Student).*
- 58. **4 Mandatory Scenarios:**
 - 1. *Token Metering Ingestion:* Service ingests token consumption events from simulated inference runs; increments student and course buckets.
 - 2. *Soft Warning Threshold (80%):* Student exceeds 80,000 of 100,000 daily token budget; system emits an advisory warning header and notification.
 - 3. *Hard Denial Enforcement (100%):* Student hits 100% quota ceiling; subsequent inference requests are rejected with a structured quota breach report.

4. *Course Budget Allocation & Reset*: Administrator allocates 5,000,000 tokens to CS-301; system configures automatic midnight quota resets.
 59. **Standalone Prototype**: Quota Management Console & Real-time Usage Ledger with simulated token traffic generator.
 60. **Mandatory Edge Case**: Concurrency race condition where two simultaneous multi-thousand token streaming requests are initiated when only 500 tokens remain in the quota bucket.
-

Module 9: Night Training Scheduler

61. **Tier**: Advanced
 62. **Core Problem**: Deep learning training jobs (fine-tuning, LoRA) occupy GPUs for hours, making them incompatible with daytime interactive teaching. The university establishes a policy: student training jobs may only execute between 22:00 and 06:00. The system must queue training submissions, validate job specs, park jobs until the night window opens, execute them under priority rules, and terminate uncompleted jobs before morning classes.
 63. **Named Protagonists**: *Kavita (Senior Research Student); Night Daemon Controller*.
 64. **4 Mandatory Scenarios**:
 1. *Daytime Submission & Queuing*: Kavita submits a LoRA training job at 14:30; scheduler accepts, validates Docker specs, and parks job in QUEUED_PENDING_WINDOW.
 2. *Night Window Activation (22:00)*: System clock strikes 22:00; scheduler awakens, scans queue, selects nodes, and transitions jobs to RUNNING.
 3. *Morning Hard Eviction (05:45)*: A long training job approaches the 06:00 morning deadline; scheduler sends SIGTERM, saves checkpoint, and frees node.
 4. *Priority Queue Preemption*: High-priority thesis submission preempts a low-priority exploration job during the night window.
 65. **Standalone Prototype**: Scheduler State-Machine Simulator with fast-forward time controls (simulate 24h in 60s) and job status board.
 66. **Mandatory Edge Case**: Job checkpointing failure during morning eviction: system must safely cordon the node, notify the user, and release GPU allocations.
-

Module 10: Distributed Training Job Planner

67. **Tier**: Advanced
68. **Core Problem**: Advanced machine learning coursework requires multi-GPU distributed training (e.g., PyTorch DDP). With 31 single-GPU machines and one dual-GPU server, scheduling a 4-worker distributed job requires validating inter-node network topology, GPU VRAM homogeneity, port allocation, and total feasibility. The planner must turn a high-level training request into an actionable, validated multi-node resource allocation.

69. **Named Protagonists:** *Dr. Chen (Deep Learning Instructor); Leo (Graduate Student).*
70. **4 Mandatory Scenarios:**
1. *Feasible Plan Generation:* Student requests 4 workers $\times$ 1 GPU for PyTorchJob; planner verifies 4 healthy nodes, assigns master/workers, allocates rendezvous ports.
 2. *Heterogeneous VRAM Rejection:* Student requests 4 workers $\times$ 48 GB VRAM; planner evaluates fleet, detects only 2 such GPUs exist, and outputs an explainable infeasibility proof.
 3. *Topology-Aware Placement:* Planner places a 2-worker job on the single dual-GPU server (server-gpu-01) to exploit PCIe/NVLink speed rather than cross-network Ethernet.
 4. *Worker Failure & Gang-Scheduling Invalidation:* During planning, 1 of 4 required nodes fails its health check; gang scheduler cancels partial allocation and returns failure.
71. **Standalone Prototype:** Kubeflow PyTorchJob Spec Generator & Topology Validation Engine with interactive resource graph.
72. **Mandatory Edge Case:** Deadlock hazard in gang-scheduling: multiple distributed jobs compete for overlapping subsets of nodes without any job reaching its full worker quorum.
-

Module 11: AI Infrastructure Observability Console

73. **Tier:** Intermediate
74. **Core Problem:** Infrastructure engineers and department heads lack a unified pane of glass into the cluster. They cannot tell if machines are idle, models are unresponsive, or quotas are saturated. The system requires an observability console that aggregates Prometheus-style metrics (GPU utilization, memory used, token throughput, request latency, active queues) and displays actionable health dashboards and operational alerts.
75. **Named Protagonists:** *Liam (Lead Cloud & Systems Engineer); Department Management.*
76. **4 Mandatory Scenarios:**
1. *Real-Time Telemetry Stream:* Console ingests metrics from 32 simulated nodes; renders heatmaps of GPU temperature and VRAM utilization.
 2. *Token Spike & Latency Alert:* System detects an abrupt 10x spike in prompt tokens causing P_{99} latency > 8 s; triggers automated visual alarm.
 3. *Unhealthy Node Drill-Down:* Node ws-gpu-22 stops reporting heartbeats; engineer drills down into timeline, error logs, and last known job.
 4. *Executive Utilization Reporting:* Console generates a 7-day academic usage summary (total GPU hours, token consumption by subject, idle capacity).
77. **Standalone Prototype:** Observability Dashboard with real-time metric graphs, live alerts list, and simulated metrics generator.

78. **Mandatory Edge Case:** Metric collection storm: telemetry generator floods console with 5,000 events/sec; dashboard applies client-side throttling and downsampling without crashing.
-

Module 12: AI Operations Copilot

79. **Tier:** Advanced
80. **Core Problem:** Lab operators spend hours diagnosing user complaints (“*Why was my job cancelled?*”, “*Why can’t CS101 access Llama-3?*”). The university needs an agentic Operations Copilot that receives natural language operational inquiries, queries simulated system state and institutional policy files, and produces cited, deterministic diagnostic explanations and recommended operational runbooks.
81. **Named Protagonists:** *Nadia (Frontline Lab Helpdesk Specialist); AI Ops Copilot Assistant.*
82. **4 Mandatory Scenarios:**
1. *Access Denial Diagnosis:* Query: “*Why was student Alex denied access to model deepseek-coder?*” Copilot retrieves user role and catalog policy, outputting cited explanation.
 2. *Night Schedule Inquiry:* Query: “*Why can’t Group 4 run their fine-tuning job right now?*” Copilot checks clock (15:00) and Night Policy rules, outputting explanation and next open window.
 3. *Node Triage Runbook Generation:* Query: “*Node ws-gpu-11 is cordoned with code ERR_THERMAL_88. What should I do?*” Copilot produces a step-by-step physical fan/airflow inspection runbook.
 4. *Safety Guardrail on Autonomous Actions:* User prompts: “*Delete all pending jobs for CS201.*” Copilot refuses direct destructive execution, requiring explicit administrative confirmation and dry-run output.
83. **Standalone Prototype:** Conversational Operations Console with integrated state inspection viewer, tool-call audit log, and citation cards.
84. **Mandatory Edge Case:** Hallucination prevention guardrail: Copilot receives a question regarding an imaginary user or non-existent policy; must output a clean “Entity Not Found in State” rather than inventing a plausible operational narrative.